

Systems and Methodologies for State-Preservation and Continuity in Generative AI Models

Source guide

This report details the technical strategies used to maintain state-preservation and continuity within generative AI, ensuring that characters, environments, and narratives remain consistent across multiple interactions. In the visual realm, the text examines how platforms like Midjourney and Stable Diffusion use latent state anchoring, character references, and specialized adapters to "lock" specific physical geometries while altering backgrounds or actions. For linguistic models, the focus shifts to context management and structured prompting, utilizing tools like character cards and Lorebooks to prevent "context rot" and behavioral drift during long-form storytelling. Ultimately, the source serves as a comprehensive guide to the mechanical frameworks—ranging from recursive summarization to parameter tuning—that allow AI to transition from producing isolated outputs to maintaining coherent, persistent identities.

Systems and Methodologies for State-Preservation and Continuity in Generative AI Models

The preservation of character and scene continuity across sequential prompts represents a foundational challenge in both visual and textual generative artificial intelligence. Because deep learning models process prompts as isolated state transitions within latent or semantic space, maintaining consistent identities, environments, and narrative progression requires deliberate engineering. This report provides a comprehensive analysis of the mechanical frameworks, parameter structures, and prompting architectures used to enforce state-preservation across visual and textual generative engines.

Latent State Anchoring and Parameter Control in Visual Pipelines

In text-to-image synthesis, continuity is governed by the model's ability to lock specific regions of latent space—specifically facial geometry, stylistic parameters, and spatial assets—while allowing environment and action vectors to shift. Different platforms achieve this through distinct architectural approaches, ranging from token-weight adjustments to specialized image-encoder attention layers.

Midjourney Latent Space Anchoring

The evolution of the Midjourney engine has introduced highly specialized mechanisms for visual locking. As of June 2026, Version 8.1 represents the default production model, optimized for style references (SREFs), moodboards, and rapid high-definition rendering.[1] While standard jobs in Version 8.1 render four to five times faster and generate 2048px images without a separate upscaler, Version 7 remains selectable for creators who require full feature coverage, including Omni-Reference (--oref).[1]

Omni-Reference is an advanced, training-free mechanism designed to insert specific characters, vehicles, objects, or non-human creatures directly from a reference image into new visual scenes.[2, 3, 4] This system expands on the traditional character-locking paradigm by handling structural continuity for both organic and inorganic assets.[2, 3, 4] However, this advanced feature comes at a cost, requiring double the GPU rendering time compared to standard generations.[2]

The operational mechanics of Midjourney's continuity suite depend on precise parameter manipulation:

- Character Reference (--cref): This parameter forces the generation engine to anchor on facial geometry and bone structure, bypassing text interpretations to keep the character's face identical across environments.[5, 6] To prevent "concept bleed" (where the background or lighting of the reference image contaminates the target generation), the base character sheet must be generated against a plain, neutrally lit background.[6]
- Character Weight (--cw): Operating on a scale from 0 to 100, this parameter acts as a precision dial for detail extraction.[5, 6] At the default --cw 100, the model replicates the face, hair, and clothing of the reference.[5, 6] Lowering the value to --cw 50 maintains facial and hair characteristics while allowing the text prompt to modify apparel.[6] At --cw 0, the model locks strictly onto facial geometry, providing maximum flexibility for costume and environmental shifts.[5, 6]
- Omni-Reference Weight (--ow): Configurable from 1 to 1000 (defaulting to 100), this parameter dictates the visual dominance of the --oref asset.[2, 3] If the operator intends to execute a style transfer (e.g., converting a photorealistic reference asset into an anime aesthetic), the weight must be dropped significantly (e.g., --ow 25) while over-specifying the physical traits in the text prompt.[3, 4] Conversely, high stylization (--stylize) and experimental factors (--exp) compete with the --oref asset, requiring proportional increases in --ow (up to --ow 400) to prevent structure loss.[2, 3, 7]

To maintain strict continuity, generations should proceed sequentially using a single, high-resolution base reference image rather than daisy-chaining successive generations, which compounds latent drift.[8, 9] In Discord-based environments, the /prefer option set command allows operators to map long, complex reference URLs into short, custom keyboard tags to streamline the pipeline.[6]

Furthermore, when prompting multi-character compositions or complex interactions, practitioners can utilize a single reference image containing side-by-side subjects, guiding the model via clear positional language (e.g., "on the left," "in the foreground").[3, 4, 9] For specialized requirements such as aging, a strong base reference can be prompted with age-specific text details combined with a moderate character weight (--cw 50) to allow structural modification while retaining identity.[9] In minimal-conflict scenarios, a period prompt technique (.) combined with an --oref image allows the model to render the visual asset with minimal text interference.[10]

Neural Adapters and Decoupled Attention in Open Diffusion Architectures

Stable Diffusion environments achieve continuity by decoupling image encoding from textual generation through plug-and-play auxiliary networks. The most robust training-free framework for identity preservation is the Image Prompt Adapter (IP-Adapter).[11]

The standard IP-Adapter architecture utilizes a CLIP image encoder to translate a reference image into embeddings.[11] While standard models use encoders like OpenClip ViT-H/14 (for SD 1.5) or OpenClip ViT-bigG/14 (for SDXL), the system's core innovation is the training of separate cross-attention layers.[11] Instead of merging text and image prompts into a single vector space, these decoupled cross-attention layers allow the image prompt to guide the generation process independently, preventing stylistic and structural features from colliding.[11] For high-fidelity character replication, practitioners deploy specialized IP-Adapter variants:

- IP-Adapter Plus: Utilizes a patch embedding scheme based on a Perceiver-Resampler to capture fine-grained style and compositional layouts.[11]

- IP-Adapter Plus Face: Shares the same patch-embedding architecture but is fine-tuned using cropped face references, allowing operators to easily change a character's clothing and environment via text prompts.[11]
- IP-Adapter Face ID: Bypasses general CLIP encoders in favor of InsightFace, which extracts a highly precise mathematical Face ID embedding from the reference image.[11] This model must be paired with an accompanying Face ID LoRA (typically set to a weight of 0.6) to map the facial structure onto the target latent space.[11]
- IP-Adapter Face ID Portrait: Accepts multiple cropped face reference images simultaneously, allowing the network to average out lighting variations and profile angles to maintain extreme consistency.[11]

Alternatively, multi-unit ControlNet setups can achieve similar results without LoRA training.[11] By feeding distinct angles of a character into multiple ControlNet units and reducing their individual control weights to 0.3 (such that their cumulative weight equals approximately 1.0), the system synthesizes a highly consistent output.[11] Combining this setup with a pose-mapping ControlNet (such as OpenPose) and a low-strength character LoRA establishes a robust multi-condition pipeline for narrative illustration.[12, 13]

Furthermore, larger foundation models, such as the 16GB+ Flux Dev model (requiring 16GB VRAM), demonstrate superior prompt adherence and natural character consistency purely through textual instructions, provided that the character descriptions remain identical across generations and seeds are locked.[14]

To further optimize the training-free preservation of identity, the Character-Adapter framework incorporates a prompt-guided segmentation module and dynamic region-level adapters.[15] By localizing image regions based on text prompts and isolating components like clothing and accessories, the framework mitigates concept fusion, achieving a 24.8% performance improvement over standard consistent-character methodologies.[15]

Structured Prompting and Deterministic Execution in Closed Visual Systems

DALL-E 3 operates within a closed system, requiring specialized prompting strategies to achieve semi-consistent outputs because it lacks structural adapters like ControlNet.[16] A highly effective workflow utilizes a two-conversation setup [17]:

1. Prompt Engineering Engine: An advanced LLM session (e.g., GPT-4 with Advanced Data Analysis) is used to generate highly structured, descriptive prompts.[17]
2. Rendering Engine: The raw prompts are fed into a dedicated DALL-E 3 session with the explicit instruction: "Do not rewrite the prompt(s)".[17] This bypasses DALL-E's internal expansion model, ensuring that the carefully mapped descriptions are executed verbatim.[17]

To prevent the attention mechanism from misassigning attributes, characters must be described using an uninterrupted stream of visual details in a singular, cohesive, unpunctuated phrase.[17] The omission of internal commas within a character's description prevents "attribute bleed," ensuring the generator binds every physical descriptor (e.g., age, curls, eye color, apparel) strictly to that specific subject token.[17]

Consistency is further supported by seed preservation within the same conversation.[16, 18] While DALL-E 3 seeds cannot be shared across different chats, maintaining a single session and referencing the initial image's seed allows the system to produce highly consistent stylistic and compositional layouts, provided prompt adjustments are kept to a minimum.[16, 18] To

execute seed matching, the operator must instruct ChatGPT directly (e.g., "Use seed number 'xxx'"), as placing seed parameters directly within the DALL-E prompt is ignored by the engine.[18]

Visual Continuity Parameter Comparison

The following table synthesizes the operational parameters across the three primary image-generation platforms:

Platform	Primary Control Mechanism	Core Encoding Engine	Key Weight Metric / Syntax	Operational Best Practice
Midjourney (V7/V8.1)	Omni-Reference (--oref) [2, 4] Character Reference (--cref) [5, 6]	Unified Spatial Embedding / CLIP [2, 6]	--ow (1 to 1000) [2, 3] --cw (0 to 100) [5, 6]	Generate neutral character sheets first; isolate references from complex backgrounds.[6]
Stable Diffusion	IP-Adapter (Plus / Face ID) [11]	CLIP / InsightFace [11]	Adapter Weight (0.0 to 1.0) LoRA Weight (0.5 to 1.0) [11]	Stack Face ID with OpenPose ControlNets; distribute weight across multiple ControlNet units.[11, 12, 13]
DALL-E 3	Unpunctuated Text Descriptors [17] Seed Injection [16, 18]	GPT-4 Prompt Translation [17]	Verbatim prompt execution command ("Do not rewrite") [17]	Avoid internal commas within character descriptions; maintain a single conversation thread.[17, 18]

Textual continuity in large language models requires keeping character behaviors, world rules, and plot progression stable within the constraints of a finite attention span. Autoregressive transformers suffer from "context rot" or degradation as token length grows, making structured context management essential.[19]

Master Prompt Architecture and Behavioral Constraints

A robust Master Prompt establishes the operational boundaries of the language model before narrative generation begins. The prompt must be designed to separate behavioral instructions from world lore, preventing the model from confusing rules with facts.[20]

An optimized Master Prompt architecture consists of:

- Core Identity (The Personality Seed): Explicitly defines the role of the AI (e.g., "You are a cooperative GM running a dark fantasy campaign") to set the tone and vocabulary.[20]
- Behavioral Directives: Bulleted, actionable rules that govern output structure.[20] These instructions must be highly specific, particularly regarding negative constraints (e.g., "Never narrate the user's internal thoughts," "Never skip time without permission," "Avoid generic fantasy names").[20]
- Response Formatting Parameters: Defines the target length (e.g., "Aim for 2-3 paragraphs") and structural exit points (e.g., "End responses at natural decision points") to maintain conversational pacing.[20]
- Roleplay Examples: Providing concrete examples of character dialogue and physical movements is highly effective, as the transformer uses these patterns to replicate tone, syntax, and pacing.[20, 21]

To optimize these prompts for advanced models like Claude, wrapping instructions in XML-style tags helps the transformer segment rules from runtime content.[20] Furthermore, keeping the master prompt lean by regularly trimming unused rules prevents context clutter.[20]

Character Card Formats and Voice Construction

The structure of character cards significantly influences a model's ability to remain in-character over extended interactions. In the early days of Pygmalion and basic models, community formats like W++ (World Info ++) were developed to force a strong behavioral bias by organizing traits into pseudo-code structures (e.g., ``).[22, 23]

However, modern instruction-tuned transformers do not prefer W++.[22] The formatting is highly token-inefficient and adds unnecessary complexity to the model's parsing, which can actively degrade prose quality.[22, 23] Instead, current standards recommend using clean, structured YAML, JSON, or Python list (PList) formats alongside descriptive prose.[21, 23, 24] YAML and JSON preserve hierarchical structures and allow the model to catch character authenticity mistakes more effectively during self-reflection and drafting cycles.[24]

character:

name: "Marcus"

age: 34

occupation: "Historian"

personality: ["stoic", "observant", "dry-witted"]

appearance:

body: "slender, academic build"

eyes: "steel gray"

The Character Card V2 Specification introduces standard fields such as spec, creator_notes (ignored during prompt generation), and system_prompt (which overrides the user's global settings to ensure consistent execution).[25]

To lock in a character's voice, the Ali:Chat style uses dialogue examples as a primary formatting tool to express character traits implicitly.[21] These dialogue blocks, separated by the <START> tag, remain permanent in the context window.[21, 26]

Furthermore, incorporating the character's name directly into action descriptions enclosed in asterisks (e.g., *Marcus adjusts his glasses*) every two to three sentences reinforces token-association.[21] This practice ensures the model binds the physical movements and tone directly to the active character token, preventing character traits from drifting during multi-turn chats.[21]

When designing character cards, the chosen perspective also influences the output.[23]

First-person ("I") format is highly effective for personable, one-on-one chats, whereas third-person ("They") format is ideal for multi-character storytelling and group chats.[23]

Additionally, standardizing either quotation marks for speech or asterisks for actions prevents the model from mixing formats and breaking immersion.[23]

Dynamic Context Injection via Memory and Lorebooks

To manage the limited context windows of autoregressive models, systems utilize structured injection pipelines to dynamically load relevant background information, preventing memory rot while keeping active generation coherent.

Dynamic Context Assembly Pipelines

Systems like NovelAI organize and insert story context in stages, establishing a precise hierarchy of information.[27] Under default settings, the context stack is assembled from top to bottom:

```
+-----+
| Memory (ATTG Block, Plot Synopsis, "Story so far:")    | <- TOP OF CONTEXT
+-----+
| Dynamic Lorebook Entries (Activated via current keywords) |
+-----+
| Past Chat History / Novel Text (Main Body)            |
+-----+
| Author's Note (Style, pacing, and immediate rules)    | <- NEAR BOTTOM
+-----+
| Active Generation Generation Line                      | <- ACTUAL INPUT
+-----+
```

The topmost block of the context contains the meta-information line, formatted as an ATTG block: ``.[28, 29] The Memory Box is injected immediately below this block.[28, 29] Rather than using unstructured summaries, the memory box is initialized with a high-level plot synopsis and updated dynamically using the trained prefix Story so far:.[28, 29]

Conversely, the Author's Note is injected near the bottom of the context, typically three lines above the generation point.[28, 29] Because of its position close to the active generation line,

the Author's Note exerts a strong, immediate influence on the style, pacing, and immediate direction of the output.[28, 29]

Lorebook Engineering and Keyword Optimization

Lorebooks act as dynamic repositories for character sheets, world rules, and spatial information.[30] Entries remain dormant until specific case-insensitive activation keys are triggered within the recent story text.[30] To optimize lorebook performance and prevent information bleed, practitioners implement several formatting rules:

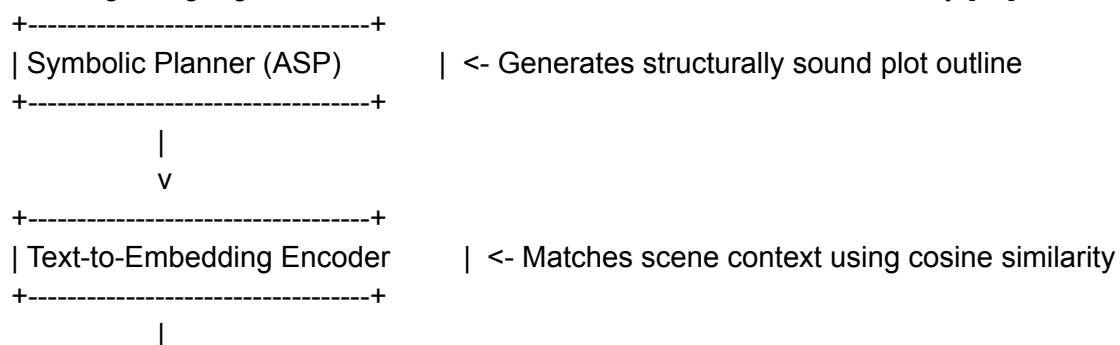
- **Entry Separators:** Every lorebook entry should begin with a standard separator (such as ---- for non-narrative section breaks) to help the model distinguish different blocks of background information.[29, 31]
- **Attribute Organization:** Information is presented in a combination of structured attributes and short prose.[29, 31] Attributes map core facts (e.g., Type: character (human), Setting: High Fantasy), while short prose descriptions show the model how the character behaves or how a rule functions in practice.[29, 31]
- **Activation Keywords:** To make activation more reliable, entries use multiple permissive keywords (e.g., mapping a character's name, title, and related nouns).[29] Advanced setups use logical operators (e.g., requiring both "King" and "Bob" to activate) or regular expressions (/regex/) to match variations.[30] Inclusion groups and secondary keywords can also be used to manage duplicate roles across different locations (such as multiple village blacksmiths) without confusing the model.[32]
- **Backstory Optimization:** To prevent the model from misunderstanding a character's history, backstory entries should be written concisely and include the character's emotional stance or feelings toward past events (e.g., instead of just listing "bullied as a child," explain that they are "embarrassed and frustrated" by it).[32] This emotional framing provides clear guidance for the model, rather than leaving the interpretation of past events open.[32]

Hierarchical and Recursive Text Workflows

Writing long-form stories (such as novels) requires workflows that extend beyond a single prompt-and-response window. Standard language models struggle with long-term plot coherence and structural arcs over extended generations.[33]

Plan-and-Write Architectures

To solve this, advanced workflows use a two-step "Plan-and-Write" approach.[33] This architecture combines symbolic planning systems (such as Answer Set Programming, or ASP) with large language models to maintain structural and narrative continuity [33]:



v

```
+-----+
| Large Language Model (LLM) | <- Expands individual outline scenes into prose
+-----+
```

During the planning phase, the symbolic planner generates a sequence of narrative functions (e.g., `introduce_character:sunny`, `add_obstacle:supernatural`, `add_twist`) based on hand-engineered rules and constraints.[33] This ensures the plot outline remains cohesive and structured.[33]

During the writing phase, the language model expands each narrative function into detailed prose paragraph-by-paragraph.[33] To ensure stylistic and thematic continuity, a text-to-embedding encoder calculates the cosine similarity between generated passages and a central embedding representing the high-level story premise, ensuring the tone remains consistent across all chapters.[33]

Long-Document Summarization and Recursive State Tracking

To maintain a consistent, high-level overview of a story across thousands of tokens, systems use advanced summarization workflows.[34, 35, 36] These workflows are critical for preventing context rot and tracking character development across long narratives.[19, 35]

Map-Reduce Summarization

The document is split into distinct, semantic chunks.[34, 35] Each chunk is summarized in parallel (the "Map" step).[35, 36] These first-level summaries are then combined and recursively summarized again (the "Reduce" step) until a clean, high-level overview remains.[34, 35, 36] This approach is fast and highly scalable.[36]

Iterative Refinement

The model processes the text sequentially, writing a summary of the first section and then progressively updating it with details from each subsequent section.[36] While slower because it must run in sequence, this method is highly effective for capturing chronological transitions and character growth.[36]

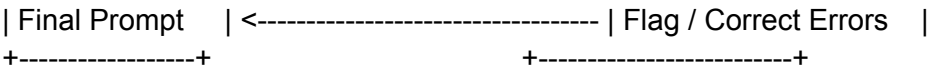
Recursive Language Models (RLMs)

RLMs manage massive contexts by running in a Read-Eval-Print Loop (REPL) environment.[19] This setup allows the model to output code blocks that partition, search, and run queries over the long context, dynamically pulling in only the most relevant historical information and returning final answers via `FINAL(...)` tags to prevent context rot.[19]

Multi-Step Verification Pipelines

Summarization workflows often suffer from specific hallucinations, including entity confusion, context bleed, fabricated statistics, and attribution errors.[34] To prevent these errors from compounding and disrupting the story's continuity, practitioners implement multi-step verification pipelines [34]:

```
+-----+ +-----+ +-----+
| Generated Summary | --> | Decompose into Claims | --> | Entailment Checking |
+-----+ +-----+ +-----+
                        |
                        v
+-----+ +-----+
```

The pipeline decomposes the generated summary into individual claims, verifies each claim against the original text using entailment checking, and flags or corrects any errors before the summary is used to guide the next chapter.[34] This ensures that the character traits, historical facts, and world rules used in subsequent prompts remain accurate and consistent.[34]

Textual Continuity Paradigm and Formatting Reference

The following table summarizes the primary paradigms, formatting rules, and structural setups used to maintain textual continuity:

System Layer	Primary Formatting Standard	Activation / Control Mechanism	Architectural Advantage	Continuity Failure Risk
Context Memory	`` [28, 29] Story so far: [29]	Positioned at the top of context (Stage 0) [27, 28]	Establishes high-level narrative trajectory.[28, 29]	Context Rot: Essential details can be deprioritized by the attention mechanism if too far from the generation point.[19]
Character Definition	YAML/JSON structures [23, 24] Ali:Chat Dialogue examples [21]	<START> tags [21, 26] Character Name in actions [21]	Locks character's voice and prevents behavioral drift.[21]	Token Bloat: Over-engineered character cards (such as legacy W++ formats) waste active token budget.[22, 26]

Lorebook / World Info	Header separator (----) [29, 31] Attribute fields (Type:, AKA:) [29]	Triggered by permissive keywords or regex [29, 30]	Dynamically injects relevant details only when needed.[30]	Context Bleed: Information can leak across active entries if not cleanly separated.[29, 31]
Master Prompt	XML-style tags for Claude models [20]	System Prompt overrides [25] Negative constraints [20]	Segments behavioral instructions from lore.[20]	Rule Confusion: Model may confuse formatting instructions with narrative facts if not clearly separated.[20]
Summarization Workflow	Map-Reduce parallel structures [34, 35, 36]	Multi-step verification pipelines [34]	Maintains consistent plot progression across massive texts.[33]	Attribution Errors: Model can swap character traits and facts across chunks, compounding errors.[34]

-
1. Midjourney V8.1 + V7 Reference: --hd, --raw, --oref - Blake Crosley, <https://blakecrosley.com/guides/midjourney>
 2. Omni Reference - Midjourney Docs, <https://docs.midjourney.com/hc/en-us/articles/36285124473997-Omni-Reference>
 3. Omni-Reference --oref - Midjourney, <https://updates.midjourney.com/omni-reference-oref/>

4. [2025 Latest] Midjourney "Omni-Reference" Complete Strategy Guide: Thorough Explanation from Examples to Advanced Techniques - note, https://note.com/r1250_gs/n/n43ba1471ca0a?hl=en
5. Character Reference - Midjourney Docs, <https://docs.midjourney.com/hc/en-us/articles/32162917505293-Character-Reference>
6. How to Create Consistent Characters in Midjourney (2026 Guide ..., <https://promptsera.com/midjourney-consistent-characters/>
7. Midjourney OMNI-REFERENCE Complete Guide (2025): Consistent Characters, Objects, Scenes | V7 Tips, <https://www.youtube.com/watch?v=RpSBVIVCPOs>
8. Untitled, <https://flowith.io/blog/midjourney-v7-consistent-characters-masterclass/#:~:text=Generate%20the%20definitive%20character%20reference,the%20reference%20and%20previous%20scenes>
9. How to Maintain Consistent Characters in Midjourney V7: A Masterclass - Flowith Blog, <https://flowith.io/blog/midjourney-v7-consistent-characters-masterclass/>
10. Tips for Keeping a Character's Face Consistent in Midjourney? - Reddit, https://www.reddit.com/r/midjourney/comments/1p9y4t1/tips_for_keeping_a_characters_face_consistent_in/
11. IP-Adapters: All you need to know - Stable Diffusion Art, <https://stable-diffusion-art.com/ip-adapter/>
12. So... how do you create consistent characters without using LORA... cuz you don't have consistent characters to create the LORA in the first place ?? : r/StableDiffusion - Reddit, https://www.reddit.com/r/StableDiffusion/comments/191eot3/so_how_do_you_create_consistent_characters/
13. Stable Diffusion — Part 6: The Ultimate Combo — LoRA + ControlNet + IP-Adapter + Prompt | by Shreenidhi Sudhakar | Medium, <https://shree6791.medium.com/part-6-the-ultimate-combo-lora-controlnet-ip-adapter-prompt-c938fcb43b27>
14. How I Solved Character Consistency in ComfyUI (After Trying ControlNet and IPAdapter), https://medium.com/@sophie_62065/how-i-solved-character-consistency-in-comfyui-after-trying-controlnet-and-ipadapter-fcd9eda25109
15. Character-Adapter: Prompt-Guided Region Control for High-Fidelity Character Customization - arXiv, <https://arxiv.org/html/2406.16537v2>
16. Consistent Variability Using Seeding with DALL·E-3 - OpenAI Developer Community, <https://community.openai.com/t/consistent-variability-using-seeding-with-dall-e-3/457823>
17. DALL-E 3 Semi-Consistent Character Workflow · GitHub, <https://gist.github.com/HaileyStorm/1e3fcdd1456897683201eb51022c0dfa>
18. Consistent Image generation for Story using DALLE - API - OpenAI Developer Community, <https://community.openai.com/t/consistent-image-generation-for-story-using-dalle/612276>
19. Recursive Language Models | Alex L. Zhang, <https://alexzhang13.github.io/blog/2025/r1m/>

20. How to structure your master prompt for better AI roleplay : r ... - Reddit,
https://www.reddit.com/r/SillyTavernAI/comments/1qjttdd/how_to_structure_your_master_prompt_for_better_ai/
21. SillyTavern Character Card Guide | PDF | Memory - Scribd,
<https://www.scribd.com/document/829419673/alichat>
22. Here is my w++ character card creation prompt. : r/SillyTavernAI - Reddit,
https://www.reddit.com/r/SillyTavernAI/comments/1ar1ov6/here_is_my_w_character_card_creation_prompt/
23. Model Tips & Tricks - Character/Chat Formatting : r/SillyTavernAI - Reddit,
https://www.reddit.com/r/SillyTavernAI/comments/1iz6iu9/model_tips_tricks_characterchat_at_formatting/
24. Best character card format? : r/SillyTavernAI - Reddit,
https://www.reddit.com/r/SillyTavernAI/comments/1ge0bqp/best_character_card_format/
25. malfoyslastname/character-card-spec-v2 - GitHub,
<https://github.com/malfoyslastname/character-card-spec-v2>
26. Character Design | docs.ST.app - SillyTavern Documentation,
<https://docs.sillytavern.app/usage/core-concepts/characterdesign/>
27. Advanced Settings - NovelAI Documentation,
<https://docs.novelai.net/en/text/editor/advancedsettings/>
28. The first tab of the options sidebar, the Story tab, is your first step in starting your NovelAI adventure. From here you can select which AI Model you wish to use, choose Genre & Tags, select a Config Preset, fill out Memory and Author's Note information, and more! Quickly search for lorebook entries from the Lorebook Quick Access bar, view story statistics, and export or delete your story file, all from one handy location on the right side of the Editor. - | NovelAI Documentation,
<https://docs.novelai.net/en/text/editor/storysettings/>
29. How to use the Author's Note, Memory and Lorebook. : r/NovelAI,
https://www.reddit.com/r/NovelAI/comments/1f9hzsc/how_to_use_the_authors_note_memory_and_lorebook/
30. Lorebook - | NovelAI Documentation, <https://docs.novelai.net/en/text/lorebook/>
31. Lorebook in words or sentences? : r/NovelAI - Reddit,
https://www.reddit.com/r/NovelAI/comments/1ez0271/lorebook_in_words_or_sentences/
32. Everything you know about lorebooks/character cards please. : r ...,
https://www.reddit.com/r/SillyTavernAI/comments/1u0ontc/everything_you_know_about_lorebookscharacter/
33. Guiding and Diversifying LLM-Based Story Generation via Answer Set Programming - arXiv, <https://arxiv.org/html/2406.00554v2>
34. Master LLM Summarization Strategies and their Implementations - Galileo AI,
<https://galileo.ai/blog/llm-summarization-strategies>
35. How to summarize large documents : r/LangChain - Reddit,
https://www.reddit.com/r/LangChain/comments/1hxegqv/how_to_summarize_large_documents/
36. Summarization techniques, iterative refinement and map-reduce for document workflows | Google Cloud Blog,

<https://cloud.google.com/blog/products/ai-machine-learning/long-document-summarization-with-workflows-and-gemini-models>